

MetaboleAI — Guida alla pubblicazione

Guida operativa per pubblicare MetaboleAI **in modo corretto**. Due parti:

- **Parte 1 — La demo** (prototipo navigabile): mettere online i file HTML su GitHub Pages, così esiste **un link unico sempre aggiornato** da mostrare a chiunque (niente più versioni vecchie in giro).
- **Parte 2 — L'app di produzione**: deploy dello stack reale (database, backend, frontend, email, pagamenti) in ordine, con le variabili d'ambiente e la sicurezza.

Regola d'oro, valida ovunque: **le chiavi non si mettono mai nel repository né in chat**. Si inseriscono solo nei pannelli dei servizi (Neon, Render, Vercel, Brevo, Stripe). In locale si usa un file `.env` che resta *git-ignored*.

Parte 1 — Pubblicare la demo (prototipo navigabile)

I prototipi (app cliente, coach, nutrizionista, widget, sondaggio) sono file HTML autonomi. La cartella `docs/` del repo è già predisposta come sito: contiene un `index.html` che fa da **hub** con i link a tutte le schermate. Pubblicandola con **GitHub Pages** si ottiene un URL pubblico stabile.

1.1 Attivare GitHub Pages (una volta sola)

Nel repository su GitHub: **Settings** → **Pages** → **Build and deployment** → **Source: "Deploy from a branch"**, poi **Branch:** `main` e **cartella:** `/docs`, infine **Save**. Dopo circa un minuto la demo è online a:

```
https://<utente-o-organizzazione>.github.io/Metabole/
```

L'hub mostra le card verso: Prototipo cliente, App Coach, App Coach (web), App Nutrizionista, Flusso di attivazione, Sondaggio iniziale, Widget mascotte.

1.2 Tenere la demo aggiornata (il punto che prima creava confusione)

GitHub Pages serve **solo** ciò che sta dentro `docs/`. Quindi ogni volta che si modifica un prototipo nella root del repo, il file aggiornato va **copiato in** `docs/` e poi si fa `commit` + `push`: Pages si rigenera da solo in circa un minuto.

```
# esempio: aggiornare il prototipo cliente pubblicato
cp Metabole_Prototipo_Navigabile.html docs/
git add docs/ && git commit -m "aggiorna demo" && git push
```

Per verificare di vedere davvero l'ultima versione, aprire il link in **finestra anonima** (evita la cache del browser) oppure ricaricare con **Ctrl/Cmd + Shift + R**.

1.3 Attenzione alla privacy della demo

La cartella `docs/` è **pubblica**. Ci vanno **solo** i file HTML dei prototipi e gli asset (audio, icone). I documenti di business, le analisi e i file del motore **non** vanno messi in `docs/`: restano nel repository (privato) ma fuori dalla pubblicazione.

Parte 2 — Pubblicare l'app di produzione

2.1 Architettura (chi fa cosa)

Servizio	Ruolo	Cartella nel repo	Region
Neon	Database PostgreSQL	(schema in <code>backend/prisma</code>)	Francoforte (UE)
Render	Backend NestJS (API) + Cron giornaliero	<code>backend/</code> (blueprint <code>render.yaml</code>)	Francoforte (UE)

Servizio	Ruolo	Cartella nel repo	Region
Vercel	Frontend app cliente	app/	Edge UE
Vercel	Frontend backoffice (staff/admin)	backoffice/	Edge UE
Brevo	Email transazionali (verifica, reset)	—	UE
Stripe	Pagamenti e webhook	—	—
Anthropic	Assistente AI + testi notifiche	—	—

Ordine di deploy (rispettarlo): 1) Database → 2) Backend → 3) Frontend → 4) Domini e CORS → 5) Email e pagamenti. Ogni passo dipende dal precedente.

2.2 Passo 1 — Database (Neon)

Creare un progetto Neon in **region Frankfurt**. Servono **due** connection string, entrambe da copiare (mai nel repo):

- **POOLED** (contiene `-pooler` nell'host): usata a runtime dall'app → variabile `DATABASE_URL`.
- **DIRECT** (senza `-pooler`): usata solo dalle migrazioni e dal seed → variabile `DIRECT_DATABASE_URL`.

2.3 Passo 2 — Backend (Render, via Blueprint)

Su Render: **New** → **Blueprint** → **collegare il repo**. Render legge `render.yaml` e crea da solo due risorse: il **web service** `metabole-backend` e il **cron** `metabole-cron-daily` (motore + notifiche ogni mattina alle 07:00 CEST).

Poi nel pannello **Environment** del web service vanno inserite a mano le variabili contrassegnate come "da inserire":

Variabile	Cosa metterci
<code>DATABASE_URL</code>	Connection string POOLED di Neon
<code>DIRECT_DATABASE_URL</code>	Connection string DIRECT di Neon
<code>BREVO_API_KEY</code>	API key di Brevo
<code>MAIL_FROM</code>	Mittente verificato su Brevo, es. <code>Metabole <no-reply@metabole.eu></code>
<code>STRIPE_SECRET_KEY</code>	Chiave segreta Stripe (<code>sk_live...</code> in produzione)
<code>STRIPE_WEBHOOK_SECRET</code>	Signing secret del webhook Stripe (<code>whsec...</code>)
<code>AI_API_KEY</code>	API key Anthropic (console.anthropic.com)
<code>AI_MODEL</code>	(<i>opzionale</i>) modello Claude, default <code>claude-haiku-4-5</code>
<code>ADMIN_EMAIL</code>	Email dell'admin principale
<code>ADMIN_PASSWORD</code>	Password iniziale admin (min. 8 caratteri)
<code>CORS_ORIGINS</code>	Domini frontend ammessi, separati da virgola (vedi Passo 4)
<code>APP_URL</code>	URL pubblico dell'app cliente (es. <code>https://app.metabole.eu</code>)

Queste **non si toccano** (le genera Render da sole): `JWT_ACCESS_SECRET`, `FILE_ENCRYPTION_KEY` (cifatura documenti sanitari), `CRON_SECRET` (condivisa con il cron).

Al deploy Render esegue in automatico: `build + prisma generate`, poi `prisma migrate deploy + db seed`, quindi avvia l'API. **Verifica:** aprire l'endpoint di salute e controllare che risponda `status: ok` con database connesso:

```
https://metabole-backend.onrender.com/health
```

2.4 Passo 3 — Frontend (Vercel)

Due progetti Vercel distinti, entrambi già configurati (`vercel.json` + Vite):

1. **App cliente** — importare il repo, impostare **Root Directory** = `app` .
2. **Backoffice** — nuovo import dello stesso repo, **Root Directory** = `backoffice` .

Per ciascuno, nelle **Environment Variables** di Vercel impostare:

```
VITE_API_URL = https://metabole-backend.onrender.com
```

(È l'unica variabile che il frontend deve conoscere: l'URL del backend.) Dopo il deploy si ottengono due URL `*.vercel.app` di prova.

2.5 Passo 4 — Domini e CORS

Collegare i domini definitivi su Vercel (Settings → Domains) aggiungendo i record **CNAME** che Vercel indica presso il gestore DNS:

- `app.metabole.eu` → progetto **app** (cliente)
- `backoffice.metabole.eu` → progetto **backoffice**

Poi tornare su Render e allineare le variabili:

- `CORS_ORIGINS` = `https://app.metabole.eu,https://backoffice.metabole.eu` (solo protocollo + dominio, **senza** slash finale; durante la transizione si possono aggiungere anche gli URL `*.vercel.app`).
- `APP_URL` = `https://app.metabole.eu` .

2.6 Passo 5 — Email (Brevo) e pagamenti (Stripe)

Brevo: verificare il **dominio mittente** (record DNS SPF/DKIM che Brevo fornisce), creare una **API key** → `BREVO_API_KEY` , e impostare `MAIL_FROM` con un indirizzo di quel dominio. Senza API key valida il backend non blocca nulla: logga l'email invece di inviarla (utile in test).

Stripe: copiare la **chiave segreta** → `STRIPE_SECRET_KEY` . Creare un **webhook** che punti all'endpoint del backend:

```
POST https://metabole-backend.onrender.com/api/v1/payments/webhook
```

Copiare il **signing secret** del webhook (`whsec_...`) → `STRIPE_WEBHOOK_SECRET` . Il backend verifica la firma ed è idempotente.

2.7 App mobile (APK Android) — opzionale, non blocca il lancio web

L'app cliente è pronta anche come app nativa via **Capacitor**. L'APK si genera in locale con **Android Studio**; la procedura è già scritta in `docs/APK_Build_Guida.md` . Non è necessaria per andare online sul web: si può fare in un secondo momento.

Sicurezza e GDPR (da rispettare al lancio)

- **Chiavi solo nei pannelli.** Mai nel repo, mai in chat, mai in `docs/` . In locale usare `.env` git-ignored. Se una chiave è finita per errore in un file o in una conversazione, va **rigenerata** (rotazione) nel servizio corrispondente.
- **Hosting UE.** Database e backend in region **Francoforte**: requisito GDPR per i dati dei clienti.
- **Dati sanitari cifrati.** I documenti clinici sono cifrati (`FILE_ENCRYPTION_KEY`) e visibili **solo** al cliente e al suo nutrizionista; la coach non vede le note cliniche (matrice permessi già configurata dal seed).
- **Repository privato.** Il repo contiene documenti di business: tenerlo privato. Solo `docs/` è pubblico via Pages.
- **Audit log attivo** e **backup del database** (Neon) verificati.

Checklist finale (da spuntare prima di annunciare)

- [] GitHub Pages attivo: l'hub della demo si apre e i link funzionano.
 - [] Neon creato in Frankfurt; connection string POOLED e DIRECT pronte.
 - [] Render: deploy verde; `GET /health` risponde `status: ok`, database connesso.
 - [] Vercel: app cliente e backoffice online; `VITE_API_URL` puntato al backend.
 - [] Domini `app.` e `backoffice.` attivi; `CORS_ORIGINS` e `APP_URL` aggiornati.
 - [] Registrazione di prova: arriva l'email di verifica (Brevo).
 - [] Pagamento di prova a buon fine e webhook ricevuto (Stripe).
 - [] Login admin con `ADMIN_EMAIL` / `ADMIN_PASSWORD`.
 - [] Tutte le chiavi eventualmente esposte sono state ruotate.
-

In una riga

Prima si pubblica la **demo** da `docs/` con GitHub Pages (link unico sempre aggiornato); poi si va in **produzione** nell'ordine Neon → Render → Vercel → domini/CORS → Brevo/Stripe, tenendo tutte le chiavi **solo nei pannelli** e i dati in **UE**.